



UNIVERSIDAD
MARIANO GÁLVEZ

Proyecto Final Programación III: Red de pagos



Roger Felipe Sosa Osorio
1890-17-8391

Guillermo David Oliva Contreras
1890-24-21944

5to. Ciclo
Ing. Carlos Tezo
Programación III
23/05/2026

ÍNDICE

1. Introducción

2. Objetivos

2.1 Objetivo general

2.2 Objetivos específicos

3. Marco teórico

3.1 Arquitectura de microservicios

3.2 APIs HTTP/JSON

3.3 Frontend SPA (Single Page Application)

3.4 Base de datos relacional (MySQL)

3.5 Despliegue en la nube (Azure App Service)

4. Desarrollo del tema

4.1 Descripción del problema

4.2 Diseño del sistema

4.3 Componentes del ecosistema

4.4 Contratos de comunicación entre servicios

4.5 Reglas de negocio

4.6 Stack tecnológico utilizado

5. Metodología

5.1 Fases del desarrollo

5.2 Herramientas de trabajo

5.3 Pruebas

6. Resultados

6.1 Microservicios entregados

6.2 Interfaces de usuario

6.3 Flujos de pago funcionales

6.4 Despliegue en producción

7. Conclusiones

8. Recomendaciones

9. Bibliografía

10. Anexos

A. Endpoints principales

B. Capturas de pantalla

C. Comandos de prueba (curl)

D. Estructura del repositorio

E. URLs del despliegue en Azure

INTRODUCCIÓN

Las instituciones financieras y las empresas de servicio (universidades, compañías telefónicas, distribuidoras eléctricas, etc.) no comparten directamente sus bases de datos. Cada organización mantiene su propia infraestructura por razones de seguridad, privacidad y autonomía operativa.

Sin embargo, los usuarios esperan poder pagar la electricidad, la mensualidad del estudio o el plan telefónico desde la banca en línea sin preocuparse por cómo se comunican esos sistemas entre sí.

La respuesta a este escenario son las APIs (Application Programming Interfaces). Una API permite que dos sistemas independientes intercambien información de forma controlada mediante un contrato bien definido, normalmente sobre HTTP/JSON.

El presente proyecto, titulado "Ecosistema de Pagos Interconectados mediante APIs", emula este entorno real. Construye cuatro sistemas independientes —un Banco y tres Empresas de Servicio (Universidad, Telefonía e Internet, Energía Eléctrica)— cada uno con su propia base de datos, su propia interfaz web y su propia API REST. La única forma en que estos sistemas pueden comunicarse es mediante peticiones HTTP/JSON, igual que en la realidad.

El proyecto integra los temas centrales del curso de Programación III: arquitectura cliente-servidor, programación asincrónica, acceso a datos con Entity Framework, exposición de servicios web con ASP.NET Core Minimal API y construcción de interfaces de usuario modernas con React y TypeScript. Además, incorpora una fase de despliegue real sobre Azure App Service y Azure Database for MySQL Flexible Server, lo cual extiende el alcance académico hacia la operación en producción.

OBJETIVOS

OBJETIVO GENERAL

Diseñar e implementar un ecosistema de pagos compuesto por cuatro microservicios independientes (un banco y tres empresas de servicio) que se comuniquen exclusivamente a través de APIs HTTP/JSON, replicando los flujos reales de banca virtual y pasarela de pago con tarjeta.

OBJETIVOS ESPECÍFICOS

- Desarrollar una API REST para el banco capaz de administrar clientes, cuentas, tarjetas, depósitos y retiros, así como exponer servicios para que terceros validen tarjetas y procesen cobros.
- Implementar tres APIs de empresa (Universidad, Telefonía, Energía), cada una con gestión propia de clientes, generación de cuotas mensuales y estado de cuenta consultable.
- Definir y documentar los contratos JSON necesarios para los tres flujos de pago: (A) Banca Virtual paga al servicio, (B) Pasarela de tarjeta desde el sitio de la empresa, (C) Pago en efectivo en el sitio de la empresa.
- Construir interfaces web modernas (SPA) para cada microservicio usando React, TypeScript y Tailwind, integradas dentro de la propia API para evitar problemas de CORS y simplificar el despliegue.
- Aplicar la regla de negocio de comisión bancaria 95% / 5%, acreditando el monto neto a una cuenta transitoria de cada empresa y reteniendo la comisión en una cuenta del banco.
- Desplegar el ecosistema en Azure App Service con base de datos MySQL en la nube, garantizando que todas las APIs queden accesibles públicamente.
- Documentar el sistema (contratos, guía de desarrollo local y guía de despliegue) para que pueda ser ejecutado y mantenido por terceros.

MARCO TEÓRICO

ARQUITECTURA DE MICROSERVICIOS

Características clave aplicadas en el proyecto:

- Cada microservicio mantiene su propio almacenamiento (banco_db, uni_db, tel_db, ee_db). Ningún servicio puede leer las tablas de otro.
- La comunicación ocurre exclusivamente vía HTTP/JSON.
- Cada microservicio puede ser desplegado, escalado y versionado por separado.

APIs HTTP/JSON

Una API HTTP expone un conjunto de endpoints (rutas) que aceptan peticiones HTTP (GET, POST, PUT, DELETE) y devuelven respuestas en formato JSON. El cliente construye una URL, envía un cuerpo JSON cuando aplica y recibe una respuesta también en JSON con un código de estado HTTP (200, 400, 404, 500, etc.).

En este proyecto se utiliza ASP.NET Core Minimal API, un estilo de programación declarativo donde cada endpoint se define en pocas líneas sin necesidad de controladores explícitos.

FRONTEND SPA (SINGLE PAGE APPLICATION)

Una SPA es una aplicación web que carga una sola página HTML inicial y, a partir de ahí, actualiza dinámicamente la interfaz sin recargar el navegador. Cada interacción con el servidor se realiza por API.

El proyecto emplea React 19 con TypeScript, empaquetado por Vite 8 y estilizado con Tailwind CSS v4 y la biblioteca de componentes accesibles shadcn/ui.

BASE DE DATOS RELACIONAL (MySQL)

Se utiliza MySQL 8 como motor de base de datos. El acceso desde .NET se realiza con Entity Framework Core y el proveedor Pomelo.EntityFrameworkCore.MySql, que permite mapear clases C# a tablas relacionales (ORM) y traducir LINQ a SQL.

Azure App Service es un servicio de plataforma (PaaS) que ejecuta aplicaciones web sin que el usuario deba administrar el sistema operativo subyacente. El proyecto se despliega como cuatro Web Apps independientes dentro del mismo plan B1 (Basic), una por microservicio.

DESARROLLO DEL TEMA

DESCRIPCIÓN DEL PROBLEMA

Se requiere construir un sistema que demuestre cómo un banco y tres empresas de servicio pueden colaborar para cobrar y acreditar pagos sin compartir bases de datos. La interacción entre los sistemas debe ser exclusivamente vía APIs y debe respetar las reglas de negocio definidas (comisión, cuenta transitoria, referencias únicas, etc.).

El sistema debe permitir tres flujos principales:

- Flujo A: el usuario inicia sesión en la banca virtual, escoge el servicio a pagar, el banco consulta a la empresa cuánto debe, descuenta el monto de la cuenta del cliente, acredita el 95% en la cuenta transitoria de la empresa, retiene el 5% como comisión, y notifica a la empresa para que marque las cuotas como pagadas.
- Flujo B: el usuario accede al portal de la empresa, ingresa los datos de su tarjeta y el portal llama al banco para validar y procesar el cobro. El banco aplica la misma regla 95% / 5%. La empresa marca las cuotas como pagadas tras la autorización.
- Flujo C: el usuario llega físicamente a la empresa y paga en efectivo. El pago se registra solamente en la empresa, sin tocar al banco.

DISEÑO DEL SISTEMA

El sistema está formado por cuatro aplicaciones web independientes, cada una con tres capas claramente diferenciadas:

Capa de presentación → SPA React (carpeta *.Web/, compilada a wwwroot/)
Capa de aplicación → ASP.NET Core Minimal API
Capa de datos → Entity Framework Core + MySQL

Cada Web App de Azure sirve API y SPA en el mismo dominio, eliminando la necesidad de configurar CORS.

COMPONENTES DEL ECOSISTEMA

Microservicio	Identificador del cliente	Color UI	Cuota típica
Banco	DPI	Azul	n/a
Universidad	Carné	Verde	Q 500
Telefonía	Número telefónico	Violeta	Q 250
Energía	Número de contador	Ámbar	Q 300

Tablas principales del banco:

- Cliente (Id, Nombre, DPI)
- Cuenta (Id, ClientId, NumeroCuenta, Saldo, Tipo: Corriente / Monetaria / Transitoria)
- Tarjeta (Id, Cuentald, Numero, Cvv, Tipo: Débito / Crédito, FechaVencimiento)
- Movimiento (Id, Cuentald, Tipo, Monto, Fecha, Referencia, Descripción)
- CuentaEmpresa (Nombre, Cuentald) — cuenta transitoria por empresa
- CuentaComisiones (Cuentald) — cuenta donde el banco acumula su 5%

Tablas principales en cada empresa:

- ClienteServicio (Id, Nombre, FechaRegistro)
- Cuota (Id, ClientId, Periodo, Monto, Pagada, FechaPago, MetodoPago, Referencia)
- MovimientoEmpresa (Id, Tipo: CuotaGenerada / PagoRecibido, Monto, Fecha, ClientId, Referencia)
- **Comisión 95% / 5%:** en todo pago de servicio cobrado por el banco, el 95% se acredita a la cuenta transitoria de la empresa y el 5% se retiene en la cuenta de comisiones del banco.
- **Cuenta transitoria por empresa:** cada empresa tiene una cuenta del tipo "Transitoria" en el banco donde recibe sus acreditaciones.
- **Retiro en cajero:** si la red NO es "Red 5B", se cobran Q 5 adicionales de comisión sobre el retiro.
- **Referencias únicas:** todo movimiento que cruza un sistema lleva una referencia generada por el banco con prefijos "BNK-", "POS-", "CAJ-", "DEP-", "RET-" o "TAR-".
- **No compartir base de datos:** cualquier intercambio de información entre dos microservicios debe ser estrictamente HTTP/JSON.

STACK TECNOLÓGICO UTILIZADO

Backend (4 APIs):

- .NET 10 SDK (10.0.103)
- ASP.NET Core Minimal API
- Entity Framework Core 9.0.15 + Pomelo.EntityFrameworkCore.MySql 9.0.0
- MySQL 8 (local) / Azure Database for MySQL Flexible Server (producción)
- Scalar.AspNetCore 2.14.x — documentación interactiva en /scalar/v1

Frontend (4 SPAs):

- Vite 8 + React 19 + TypeScript
- Tailwind CSS v4 (vía @tailwindcss/vite)
- shadcn/ui sobre Radix UI — componentes accesibles
- lucide-react (iconos), sonner (toasts), next-themes (modo claro/oscuro)

Infraestructura:

- Azure App Service — 4 Web Apps en un plan B1 Basic compartido
- Azure Database for MySQL Flexible Server — 4 bases en el mismo servidor

Repositorio (Source Code) en Github:

<https://github.com/rsosao/EcosistemaPagos>

METODOLOGÍA

FASES DEL DESARROLLO

1. Análisis del enunciado y definición de contratos de API.
2. Diseño de modelos de datos y diagrama de entidades por cada microservicio.
3. Implementación de los modelos en C# con Entity Framework Core.
4. Implementación de endpoints de cada API (Minimal API) con sus DTO correspondientes para evitar ciclos JSON en las respuestas.
5. Construcción de las SPAs en React + TypeScript, una por microservicio, con un estilo visual distintivo por dominio (banco/universidad/telefonía/energía).
6. Implementación del cliente HTTP de cada empresa hacia el banco (BancoClient) y del cliente del banco hacia las empresas (EmpresaClient).
7. Pruebas funcionales locales con MySQL y los tres flujos.
8. Documentación: contratos-api.md, desarrollo-local.md y despliegue-azure.md.
9. Despliegue en Azure App Service + Azure Database for MySQL.
10. Pruebas integrales en la nube.

HERRAMIENTAS DE TRABAJO

- Editor / IDE: Cursor IDE (basado en VS Code).
- Control de versiones: Git.
- Empaquetado: dotnet publish + npm run build.
- Cliente HTTP de pruebas: curl, Scalar (UI integrada en cada API) y la propia SPA de cada microservicio.
- Infraestructura como código: comandos "az" (CLI de Azure).

RESULTADOS

MICROSERVICIOS

- BancoApi
- UniversidadApi
- TelefoníaApi
- EnergíaApi

Cada API expone documentación interactiva en /scalar/v1.

INTERFACES DE USUARIO

Cada microservicio cuenta con una SPA propia con diseño coherente pero distinguible por color:

- Banco: dashboard con saldos, gestión de clientes, cuentas, tarjetas, cajero y pasarela POS. Incluye reportes de comisiones y movimientos.
- Universidad: alta de estudiantes por carné, generación masiva de cuotas, pago en efectivo y por tarjeta, reportes de pendientes y pagados.
- Telefonía: gestión por número telefónico, mismos flujos de cobro y reportes.
- Energía: gestión por número de contador, mismos flujos de cobro y reportes.

FLUJOS DE PAGO FUNCIONALES

Se validaron los tres flujos definidos en el enunciado:

- Flujo A — Pago desde la banca virtual del Banco hacia la Universidad, Telefonía y Energía. El banco descuenta del cliente, acredita el 95% a la empresa y retiene el 5% como comisión.
- Flujo B — Pasarela de tarjeta en el portal de la empresa, con la validación y autorización por parte del Banco.
- Flujo C — Pago en efectivo dentro del portal de la empresa, sin involucrar al Banco.

DESPLIEGUE EN PRODUCCIÓN

El ecosistema se desplegó en Azure con:

- Grupo de recursos: rg-pagos
- 4 Web Apps de Azure App Service (plan B1 Basic compartido)
- 1 servidor MySQL Flexible Server con 4 bases (banco_db, uni_db, tel_db, ee_db)

Cada Web App sirve API y SPA en el mismo dominio público, eliminando la necesidad de CORS.

DESPLIEGUE EN AZURE

Grupo de recursos: rg-pagos

Plan de App Service: B1 Basic (compartido por las 4 Web Apps)

Servidor MySQL: 1 Flexible Server con 4 bases (banco_db, uni_db, tel_db, ee_db)

SPAs (interfaz de usuario, una por microservicio):

Microservicio	URL de la SPA (UI)
Banco	https://banco-api-6442.azurewebsites.net/
Universidad	https://uni-api-21335.azurewebsites.net/
Telefonía	https://tel-api-7163.azurewebsites.net/
Energía	https://ee-api-14437.azurewebsites.net/

Documentación interactiva (Scalar):

Microservicio	URL de Scalar (docs OpenAPI)
Banco	https://banco-api-6442.azurewebsites.net/scalar/v1
Universidad	https://uni-api-21335.azurewebsites.net/scalar/v1
Telefonía	https://tel-api-7163.azurewebsites.net/scalar/v1
Energía	https://ee-api-14437.azurewebsites.net/scalar/v1

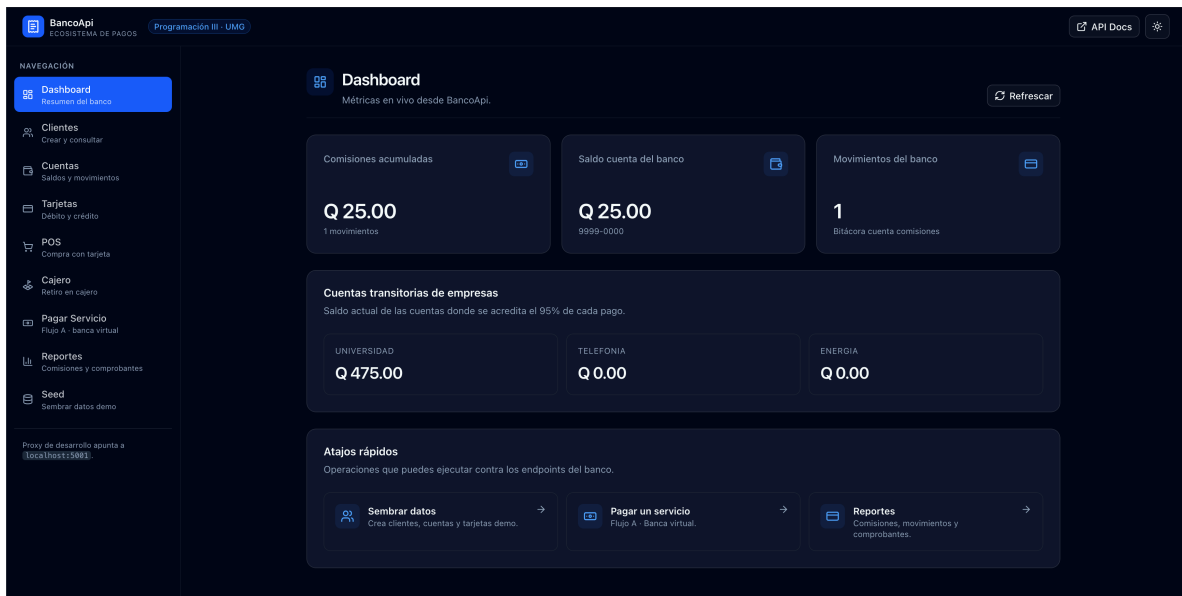
Repositorio (Source Code) en Github:

<https://github.com/rsosao/EcosistemaPagos>

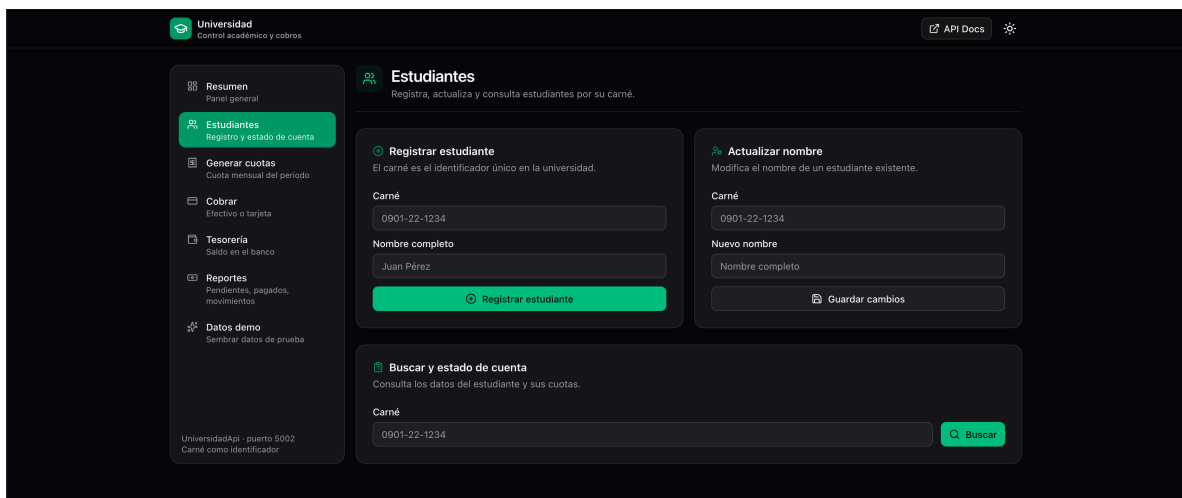
Anexos

CAPTURAS DE PANTALLA

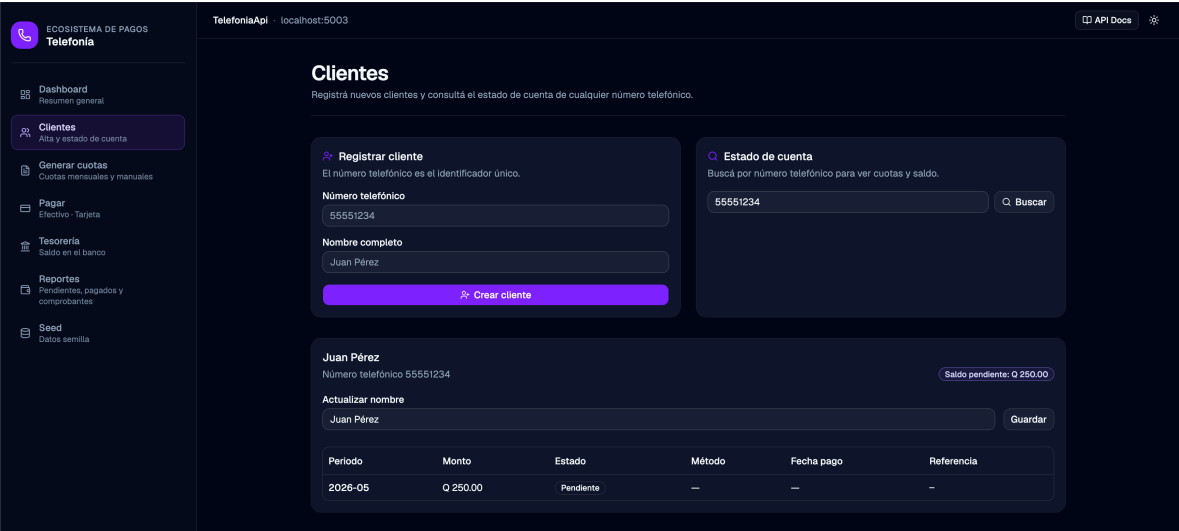
- Banco: dashboard de cuentas, gestión de tarjetas, cajero POS.



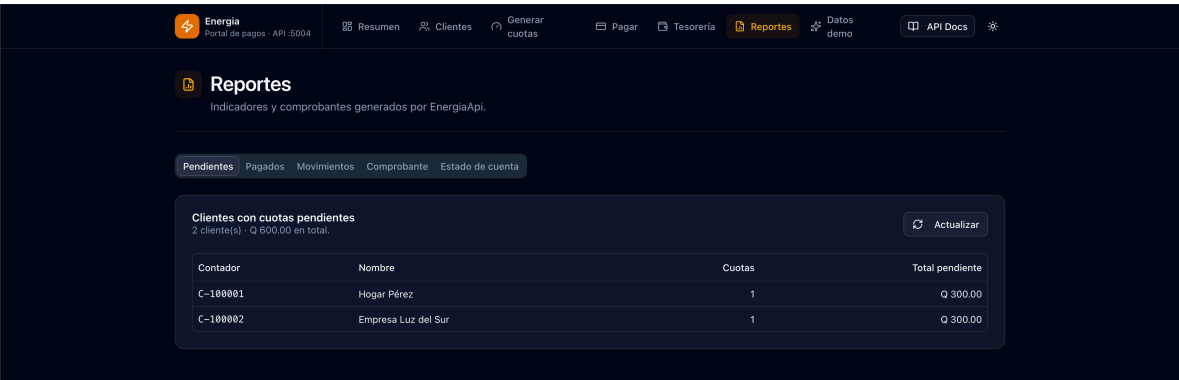
- Universidad: registro de estudiante, generación de cuotas, pago con tarjeta.



- Telefonía: estado de cuenta de un cliente, comprobante de pago.



- Energía: reporte de clientes pendientes y movimientos.



- Scalar: documentación interactiva de cada API en /scalar/v1.

